

Basic R

A practical introduction to RStudio, data transformation, import and visualization

RStudio • tidyverse • dplyr • ggplot2 • data import

Agenda

- Intro to R and Rstudio
- Intro to tidyverse
- Pizza (18.30)
- Visualisation
- Folder structures
- Importing data
- Final remarks

Before we start

- Expectations to teaching:
 - Interactive session mixed with intro lecture
 - Questions and comments are welcome!
 - Open dialogue
- Educational background?
- Installation of R, Rstudio, and tidyverse?
- Experience with R?
- https://github.com/SteffanChristiansen/basic_R
- Disclaimer

CODE

Learning roadmap for Basic R

- RStudio & the R workflow
- Objects, variables and data types
- Operators and functions
- The tidyverse ecosystem
- Pipes and readable workflows
- Data transformation with dplyr
- Summarising and counting
- Importing data
- Joins and combining data
- Visualization with ggplot2
- Good coding practices

Getting comfortable in RStudio

- Console — run commands interactively
- Scripts — write reusable R code
- Environment — inspect objects currently in memory
- Files — navigate project files
- Plots — view visualizations
- Packages — install and load packages

```
2 + 2  
x <- 10  
x * 2  
print(x)
```

```
df <- data.frame(a = c(0,2,3), b = c(0,2,6))  
  
plot(df$a, df$b)
```

Getting comfortable in Rstudio - settings

- Enable Rainbow Parentheses to make nested expressions easier to read
 - Tools -> Global options -> Code -> Display -> Use rainbow parentheses
- Avoid RStudio History to recover or inspect previous commands!

Recommended workflow

- Keep code in scripts rather than relying on Console history
- Use consistent indentation and naming for variables (e.g. snake_case)

```
# Consistent naming  
a <- 10  
b <- 20
```

```
# Avoid  
a <- 10  
B <- 10
```

Recommended workflow

- Avoid spaces in variable names; use names such as total_sales
- Be consistent with capitalization
- Use descriptive variable names
- Use comments to explain why, not every obvious step
- Use consistent folder structure
 - Consider using an RStudio Project for each analysis (Day II?)
 - Keep raw data separate from processed data
- Restart R from time to time to ensure that code runs as expected

```
# Good naming
total_sales <- 12500
average_price <- mean(price)
```

```
# Avoid
total sales <- 12500
total_sales_final2 <- 12500
```

Objects and data types

- An object stores a value that can be reused
- Assignment is commonly written with <-
- Numeric values represent numbers
- Character values represent text
- Logical values are TRUE or FALSE
- Vectors contain multiple values of the same basic type
- Data frames and tibbles contain rows and columns
- Factors are useful for categorical data

```
age <- 35
name <- "Anna"
passed <- TRUE
scores <- c(75, 82, 91)
df <- data.frame(name, age, passed)
class(age)
class(df)
str(df)
```

Operators, variables and functions

- Arithmetic: +, -, *, /, ^
- Comparison: ==, !=, >, <, >=, <=
- Assignment: <- or =
- Functions take inputs and return results
- Arguments control how functions behave
- Use ?function_name to open documentation

```
x <- 10
x + 5
x == 10
x > 5
mean(c(10, 20, 30))
round(3.14159, digits = 2)
?mean
```

Operators, variables and functions

- Arithmetic: +, -, *, /, ^
- Comparison: ==, !=, >, <, >=, <=
- Assignment: <- or =
- Functions take inputs and return results
- Arguments control how functions behave
- Use ?function_name to open documentation

```
x <- 10
x + 5
x == 10
x > 5
mean(c(10, 20, 30))
round(3.14159, digits = 2)
?mean
```

Break?

What is the tidyverse?

- A collection of R packages designed to work together
- dplyr — data transformation
- ggplot2 — visualization
- tidyr — reshaping and tidying data
- readr — rectangular text data import
- readxl — Excel files
- tibble — modern data frames
- stringr — text and categorical data

```
install.packages("tidyverse")  
library(tidyverse)  
  
# Or load an individual package  
library(readxl)
```

Counting, slicing and summarising

ToothGrowth

The Effect of Vitamin C on Tooth Growth in Guinea Pigs

Description

The response is the length of odontoblasts (cells responsible for tooth growth) in 60 guinea pigs. Each animal received one of three dose levels of vitamin C (0.5, 1, and 2 mg/day) by one of two delivery methods, orange juice or ascorbic acid (a form of vitamin C and coded as VC).

```
?ToothGrowth
```

```
df <- ToothGrowth
```

```
glimpse(df)
```

Counting, slicing and summarising

Pipes

- The pipe (`|>`) passes the result of one step into the next
- `%>%` is the traditional magrittr/tidyverse pipe
- Pipes make multi-step transformations easier to read
- Think: take data ? do something ? do something else

Useful functions

- `count()` — quickly count observations by one or more variables
- `slice_min()` — keep rows with the smallest values
- `slice_max()` — keep rows with the largest values

```
df |>
  count(supp)

df |>
  count(supp, dose)

df |>
  slice_max(len, n = 3)
```

Counting, slicing and summarising

Pipes

- The pipe (`|>`) passes the result of one step into the next
- `%>%` is the traditional magrittr/tidyverse pipe
- Pipes make multi-step transformations easier to read
- Think: take data ? do something ? do something else

Useful functions

- `count()` — quickly count observations by one or more variables
- `slice_min()` — keep rows with the smallest values
- `slice_max()` — keep rows with the largest values

How to find the two rows with shortest length?

```
df |>
  count(supp)

df |>
  count(supp, dose)

df |>
  slice_max(len, n = 3)
```

Counting, slicing and summarising

Useful functions

- `reframe()` — reduce many rows to summary statistics
 - `mean()`
 - `median()`
 - `min()`
 - `max()`
 - `sd()`
- `group_by()` + `reframe()` gives group-level statistics

```
df |>
  group_by(supp, dose) |>
  reframe(
    mean = mean(len),
    median = median(len),
    sd = sd(len)
  )
```

Counting, slicing and summarising

Useful functions

- `reframe()` — reduce many rows to summary statistics
 - `mean()`
 - `median()`
 - `min()`
 - `max()`
 - `sd()`
- `group_by()` + `reframe()` gives group-level statistics

How to get the median across all 60 samples?

```
df |>
  group_by(supp, dose) |>
  reframe(
    mean = mean(len),
    median = median(len),
    sd = sd(len)
  )
```

Core dplyr verbs

- `select()` — choose columns
- `filter()` — keep rows matching conditions
- `mutate()` — create or modify columns
- `arrange()` — sort rows

```
df |>
  select(dose)

df |>
  filter(supp == "VC") |>
  arrange(len)

df |>
  mutate(len_mm = len/1000)
```

Visualization with ggplot2 – scatter plot

- Start with `ggplot(data, aes(...))`
- Map variables to visual aesthetics
- Add layers using `+`
- `geom_point()` — scatter plot
- `geom_boxplot()` — box plot
- `labs()` adds titles and labels
- Use themes to control appearance
- Export in Rstudio or using `ggsave()`

```
df |>
  ggplot(aes(x = supp, y = len)) +
  geom_point()

df |>
  ggplot(aes(x = supp, y = len, color = factor(dose))) +
  geom_point(position = position_dodge(width = 0.5)) +
  labs(
    x = "Supplement type (VC or OJ)",
    y = "Tooth length",
    color = "Dose (milligrams/day)"
  ) +
  theme_minimal()
```

Visualization with ggplot2 – scatter plot

- Start with `ggplot(data, aes(...))`
- Map variables to visual aesthetics
- Add layers using `+`
- `geom_point()` — scatter plot
- `geom_boxplot()` — box plot
- `labs()` adds titles and labels
- Use themes to control appearance
- Export in Rstudio or using `ggsave()`

```
df |>
  ggplot(aes(x = supp, y = len)) +
  geom_point()

df |>
  ggplot(aes(x = supp, y = len, color = factor(dose))) +
  geom_point(position = position_dodge(width = 0.5)) +
  labs(
    x = "Supplement type (VC or OJ)",
    y = "Tooth length",
    color = "Dose (milligrams/day)"
  ) +
  theme_minimal()
```

Consider first example: How to plot length on horizontal and supplement on vertical axis?

Visualization with ggplot2 – scatter plot

- Start with `ggplot(data, aes(...))`
- Map variables to visual aesthetics
- Add layers using `+`
- `geom_point()` — scatter plot
- `geom_boxplot()` — box plot
- `labs()` adds titles and labels
- Use themes to control appearance
- Export in Rstudio or using `ggsave()`

```
df |>
  ggplot(aes(x = supp, y = len)) +
  geom_point()

df |>
  ggplot(aes(x = supp, y = len, color = factor(dose))) +
  geom_point(position = position_dodge(width = 0.5)) +
  labs(
    x = "Supplement type (VC or OJ)",
    y = "Tooth length",
    color = "Dose (milligrams/day)"
  ) +
  theme_minimal()
```

How to increase size of dots using `geom_point()`?

Visualization with ggplot2 – box plot

- Start with `ggplot(data, aes(...))`
- Map variables to visual aesthetics
- Add layers using `+`
- `geom_point()` — scatter plot
- `geom_boxplot()` — box plot
- `labs()` adds titles and labels
- Use themes to control appearance
- Export in Rstudio or using `ggsave()`

```
df |>
  ggplot(aes(x = factor(dose), y = len, fill = supp)) +
  geom_boxplot() +
  labs(
    x = "Dose (milligrams/day)",
    y = "Tooth length",
    color = "Supplement type") +
  theme_minimal()
```

Break?

Folder structures in projects

- Better organization: Keeps your scripts, data, results, and other files together in one dedicated folder.
 - Easy file management: RStudio automatically works within the project's folder, **reducing problems with file paths**.
 - Reproducibility: Makes it easier for others (and future self!) to run the analysis without changing file locations.
1. File
 2. New Project
 3. New Directory
 4. New Project
 5. Enter Directory name: "basic_r"

Folder structures in projects

- data
 - raw data
 - README files (including date and URL for downloaded data)
- results
 - processed (transformed, normalized, or any other processed data)
 - images
 - tables
- doc (documentation including papers, correspondence with other researchers, etc.)
- scripts – scripts (.R, .rmd, and .qmd files – see chapter 28)
- reports (.html files based on .rmd and .qmd files)

```
dir() # blank
```

```
dir.create("scripts")  
dir.create("data")  
dir.create("reports")  
dir.create("doc")  
dir.create("results")  
dir.create("results/tables")  
dir.create("results/images")  
dir.create("results/processed")
```

```
dir() # created folder structure
```

Reading data into R

- Use `getwd()` to inspect the current working directory
- Use projects or explicit paths to make file locations reproducible
- `readr` functions import common text files
 - `read_csv()`
 - `read_tsv()`
- `readxl::read_excel()` imports Excel worksheets
- Inspect imported data before transforming it
- Useful checks: `glimpse()`, `head()`, `names()`, `summary()`
- Always verify column names and data types after import

Reading data into R

Typical challenges of reading data

- Wrong path to file
- Failure to recognize column names
- Failure to recognize delimiters
- Wrong class of columns
- Missing data fields
- Inconsistent file structure
 - First 30 lines: four columns
 - Last 5 lines: one column (typically with random experimental notes)
- Same delimiters between columns and in cells (comma as decimal separator in .csv file)
 - A1: 3,0
 - B1: 3,2

The GTExPortal data

- RNA data from ~1000 deceased individuals (sampled from 54 tissues)
- Deidentified metadata including sex, age group, and death classification (Hardy scale)



Death classification based on the 4-point Hardy Scale:

- 1) **Violent and fast death** Deaths due to accident, blunt force trauma or suicide, terminal phase estimated at < 10 min.
- 2) **Fast death of natural causes** Sudden unexpected deaths of people who had been reasonably healthy, after a terminal phase estimated at < 1 hr (with sudden death from a myocardial infarction as a model cause of death for this category)
- 3) **Intermediate death** Death after a terminal phase of 1 to 24 hrs (not classifiable as 2 or 4); patients who were ill but death was unexpected
- 4) **Slow death** Death after a long illness, with a terminal phase longer than 1 day (commonly cancer or chronic pulmonary disease); deaths that are not unexpected
- 0) **Ventilator Case** All cases on a ventilator immediately before death.